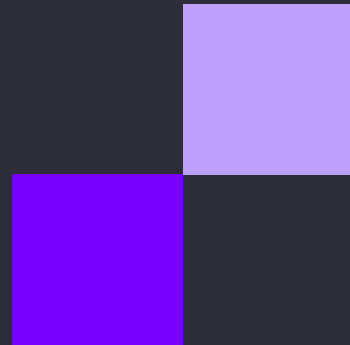




онлайн
университет



ВЫСШАЯ ШКОЛА
ЭКОНОМИКИ



SQL для начинающих

Занятие 5



Оконные функции

Типы оконных функций:

1. Агрегирующие – те же, которые мы рассмотрели ранее: SUM, COUNT, AVG, MIN, MAX
2. Ранжирующие – это те, которые показывают положение строки в выборке:
 - ROW_NUMBER – номер строки в выборке
 - RANK – ранг строки в выборке. Отличие от номера в том, что номер всегда уникален, а ранг – это номер группы, состоящей из одинаковых значений
 - DENSE_RANK – тоже ранг, но в случае одинаковых значений не пропускает следующий числовой ранг, а идет последовательно

Оконные функции

Типы оконных функций:

3. Функции смещения – возвращают значение этой же колонки из предыдущей, следующей или любой другой указанной строки

- LAG() – предыдущее значение столбца с учетом сортировки ORDER BY
- LEAD() - следующее значение столбца с учетом сортировки ORDER BY
- FIRST_VALUE() – первое значение столбца в указанной группе
- LAST_VALUE() – последнее значение столбца в указанной партии
- NTH_VALUE(n) – n-ое значение в упорядоченном наборе значений из аналитического окна

Это даёт возможность:

- Сравнивать текущее значение с ближайшими соседями (например, вычислять разницу между текущей и предыдущей записью);
- Проводить расчёты, зависящие от соседних значений, не прибегая к самостоятельному объединению таблиц (self-join);
- Реализовывать аналитические расчёты, такие как скользящие суммы, скользящие средние или накопительные итоги, когда важен контекст строки в рамках всего набора данных.

Ранжирующие функции

RANK(). Возвращает ранг строки в пределах раздела, при этом строки с одинаковым значением получают один и тот же ранг, а следующий ранг пропускается.

```
SELECT booking_id, renter_id, check_in_date,  
RANK() OVER (PARTITION BY renter_id ORDER BY check_in_date) AS booking_rank  
FROM bookings;
```

Ранжирующие функции

DENSE_RANK(). Похож на RANK(), но не пропускает ранги. То есть, если несколько строк имеют одинаковый ранг, следующий ранг будет просто на единицу больше.

```
SELECT booking_id, renter_id, check_in_date,  
DENSE_RANK() OVER (PARTITION BY renter_id ORDER BY check_in_date)  
AS dense_booking_rank  
FROM bookings;
```

Ранжирующие функции

ROW_NUMBER(). Возвращает уникальный номер для каждой строки в пределах раздела.

```
SELECT booking_id, renter_id, check_in_date,  
ROW_NUMBER() OVER (PARTITION BY renter_id ORDER BY check_in_date) AS row_number  
FROM bookings;
```

Различия в ранжирующих функциях

id	name	department	salary
1	Alice	Sales	5000
2	Bob	Sales	6000
3	Charlie	Sales	6000
4	Dave	HR	4500
5	Eve	HR	5000
6	Frank	HR	4500
7	Grace	IT	7000
8	Heidi	IT	7000

id	name	department	salary	rank	dense_rank	row_number
5	Eve	HR	5000	1	1	1
4	Dave	HR	4500	2	2	2
6	Frank	HR	4500	2	2	3

id	name	department	salary	rank	dense_rank	row_number
2	Bob	Sales	6000	1	1	1
3	Charlie	Sales	6000	1	1	2
1	Alice	Sales	5000	3	2	3

Ранжирующие функции

NTILE(). Разделяет строки на заданное количество групп (по умолчанию равных) и возвращает номер группы.

```
SELECT booking_id, renter_id, check_in_date,  
NTILE(4) OVER (ORDER BY check_in_date) AS quartile  
FROM bookings;
```


Задача 1

Пронумеровать комнаты в порядке возрастания цены внутри каждого типа комнаты. Вывести номер комнаты, ее тип, цену и колонку с названием `row_number`, в которой будет указан порядковый номер, описанный условием ранее.

Задача 1

Пронумеровать комнаты в порядке возрастания цены внутри каждого типа комнаты. Вывести номер комнаты, ее тип, цену и колонку с названием `row_number`, в которой будет указан порядковый номер, описанный условием ранее.

```
SELECT
room_number,
type_name,
price_per_night,
ROW_NUMBER() OVER (PARTITION BY type_name ORDER BY price_per_night) AS row_number
FROM rooms;
```

Задача 2

Пронумеровать комнаты каждого типа по убыванию площади без пропусков в нумерации, присвоив одинаковым по площади номерам одинаковый ранг. Вывести номер комнаты, ее тип, площадь и колонку `sqm_rank`, в которой будет указан номер из описанного выше условия

Задача 2

Пронумеровать комнаты каждого типа по убыванию площади без пропусков в нумерации, присвоив одинаковым по площади номерам одинаковый ранг. Вывести номер комнаты, ее тип, площадь и колонку `sqm_rank`, в которой будет указан номер из описанного выше условия

```
SELECT
room_number,
type_name,
room_size_sqm,
DENSE_RANK() OVER (PARTITION BY type_name ORDER BY room_size_sqm DESC) AS sqm_rank
FROM rooms;
```

Аналитические функции

LAG(). Возвращает значение из предыдущей строки в пределах раздела.

```
SELECT booking_id, renter_id, check_in_date,  
LAG(check_in_date) OVER (PARTITION BY renter_id ORDER BY check_in_date) AS previous_checkin  
FROM bookings;
```

Аналитические функции

LEAD(). Возвращает значение из следующей строки в пределах раздела.

```
SELECT booking_id, renter_id, check_in_date,  
LEAD(check_in_date) OVER (PARTITION BY renter_id ORDER BY check_in_date) AS next_checkin  
FROM bookings;
```

Функции смещения

FIRST_VALUE(). Возвращает первое значение в окне.

```
SELECT booking_id, renter_id, check_in_date,  
FIRST_VALUE(check_in_date) OVER (PARTITION BY renter_id ORDER BY check_in_date) AS  
first_checkin  
FROM bookings;
```

Функции смещения

LAST_VALUE(). Возвращает последнее значение в окне.

SELECT

 booking_id,
 renter_id,
 check_in_date,

 LAST_VALUE(check_in_date) OVER (
 PARTITION BY renter_id
 ORDER BY check_in_date ROWS BETWEEN UNBOUNDED PRECEDING
 AND UNBOUNDED FOLLOWING

) AS last_checkin

FROM bookings;

Функции смещения

При использовании функции LAST_VALUE в оконной функции SQL с предложением ORDER BY, по умолчанию рамка окна (window frame) простирается от начала раздела (partition) до текущей строки. То есть, для каждой строки функция LAST_VALUE будет возвращать значение из последней строки в текущей рамке окна, которая по умолчанию заканчивается на текущей строке.

Это означает, что без явного указания рамки окна:

```
SELECT booking_id, renter_id, check_in_date,  
LAST_VALUE(check_in_date) OVER  
(PARTITION BY renter_id ORDER BY check_in_date ) AS last_checkin  
FROM bookings;
```

Функция LAST_VALUE вернёт значение check_in_date текущей строки, так как рамка окна заканчивается на текущей строке, и более поздние даты не будут учтены.

Чтобы функция LAST_VALUE возвращала последнее значение check_in_date во всём разделе renter_id, необходимо расширить рамку окна так, чтобы она охватывала все строки в разделе. Это достигается добавлением предложения ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING

ROWS и RANGE

ROWS: Указывает диапазон строк в окне, основываясь на физическом расположении строк. Это означает, что диапазон будет определен как количество строк до и после текущей строки, независимо от их значений.

RANGE: Определяет диапазон строк на основе значений в определенном столбце. Это значит, что строки, имеющие одинаковое значение, будут входить в один диапазон.

Немного тонкостей

Рамка окна задаёт диапазон строк внутри окна, которые используются для вычисления оконной функции для каждой строки результата. Она определяется с помощью предложения ROWS или RANGE и может быть настроена различными способами.

По умолчанию, если вы указываете ORDER BY, но рамка окна не задаётся явно, то рамка окна:

Начинается с начала раздела (UNBOUNDED PRECEDING).

Заканчивается текущей строкой (CURRENT ROW).

Это означает, что для каждой строки функция будет учитывать все предыдущие строки в разделе вплоть до текущей.

Немного тонкостей

Пояснение ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING

ROWS: Указывает, что рамка окна определяется на основе физических строк (в отличие от RANGE, который работает с логическими диапазонами значений).

BETWEEN ... AND ...: Задаёт рамку окна с указанием начальной и конечной точки.

UNBOUNDED PRECEDING: Означает "неограниченно предшествующие" строки, то есть рамка начинается с первой строки в разделе.

UNBOUNDED FOLLOWING: Означает "неограниченно последующие" строки, то есть рамка заканчивается последней строкой в разделе.

Таким образом, ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING устанавливает рамку окна, которая охватывает все строки в текущем разделе для каждой строки результата. Оконная функция будет применяться ко всем строкам раздела, независимо от позиции текущей строки.

Немного тонкостей

- Предыдущие две строки и текущая:

```
ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
```

Рамка включает текущую строку и две предшествующие ей строки.

- От текущей строки до двух следующих:

```
ROWS BETWEEN CURRENT ROW AND 2 FOLLOWING
```

Рамка включает текущую строку и две следующих за ней строки.

- Все строки в разделе после текущей:

```
ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING
```

Задача 3

Определить скользящее среднее количество бронирований за последние 3 дня для каждого арендатора.

Задача 3

Определить скользящее среднее количество бронирований за последние 3 дня для каждого арендатора.

```
SELECT booking_id, renter_id, check_in_date,  
COUNT(*) OVER (PARTITION BY renter_id ORDER BY check_in_date  
ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)  
AS moving_average_count FROM bookings;
```

Задача 4

Определить общее количество бронирований для арендаторов, у которых даты регистрации находятся в диапазоне 420 дней.

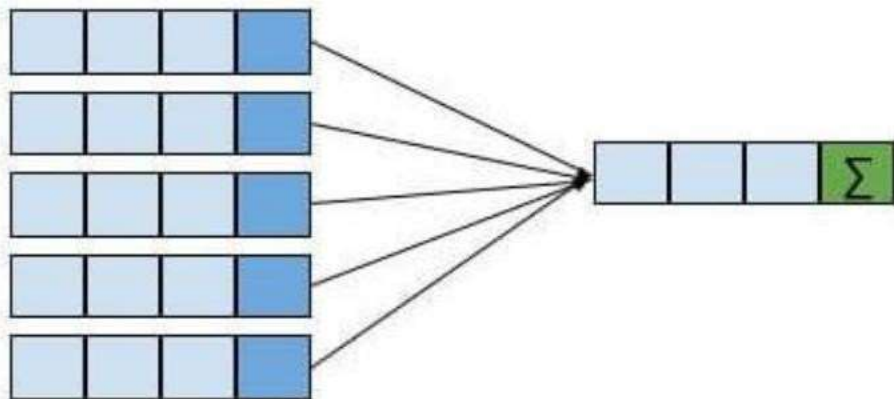
Задача 4

Определить общее количество бронирований для арендаторов, у которых даты регистрации находятся в диапазоне 420 дней.

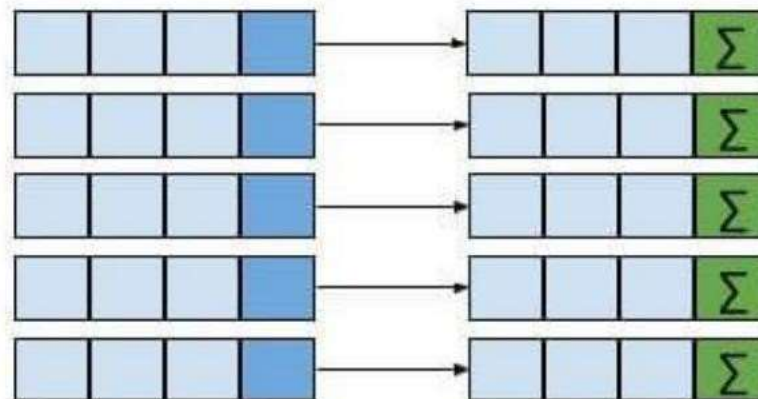
```
SELECT
    booking_id,
    renter_id,
    check_in_date,
    COUNT(*) OVER (PARTITION BY renter_id ORDER BY check_in_date
RANGE BETWEEN INTERVAL '420 days' PRECEDING AND CURRENT ROW) AS total_bookings_in_range
FROM bookings;
```

Агрегатные функции и оконные

Aggregate Functions (SUM, AVG, etc.)



Window Functions



Задача 5

Выдайте ранжирование бронирований по дате регистрации (по `check_in_date`) для каждого `renter_id`.

	booking_id integer	renter_id integer	room_number integer	check_in_date timestamp without time zone	booking_rank bigint
1	10796	1	4	2021-01-29 23:23:30	1
2	1526	2	28	2018-06-04 19:15:54	1
3	14937	3	33	2022-04-07 15:43:27	1
4	2238	4	2	2018-08-16 20:27:52	1
5	12749	4	23	2021-08-21 17:09:52	2
6	5914	5	48	2019-09-06 18:00:20	1
7	6615	5	32	2019-11-20 18:41:13	2
8	11492	5	27	2021-04-12 15:45:29	3

Задача 5

Выдайте ранжирование бронирований по дате регистрации (по check_in_date) для каждого renter_id.

SELECT

booking_id,
renter_id,
room_number,
check_in_date,

RANK() **OVER** (**PARTITION BY** renter_id **ORDER BY** check_in_date) **AS** booking_rank

FROM bookings;

	booking_id integer	renter_id integer	room_number integer	check_in_date timestamp without time zone	booking_rank bigint
1	10796	1	4	2021-01-29 23:23:30	1
2	1526	2	28	2018-06-04 19:15:54	1
3	14937	3	33	2022-04-07 15:43:27	1
4	2238	4	2	2018-08-16 20:27:52	1
5	12749	4	23	2021-08-21 17:09:52	2
6	5914	5	48	2019-09-06 18:00:20	1
7	6615	5	32	2019-11-20 18:41:13	2
8	11492	5	27	2021-04-12 15:45:29	3

Задача 6

Определите, когда был бы следующий check_in для каждого renter_id

	booking_id integer	renter_id integer	check_in_date timestamp without time zone	next_check_in timestamp without time zone
1	10796	1	2021-01-29 23:23:30	[null]
2	1526	2	2018-06-04 19:15:54	[null]
3	14937	3	2022-04-07 15:43:27	[null]
4	2238	4	2018-08-16 20:27:52	2021-08-21 17:09:52
5	12749	4	2021-08-21 17:09:52	[null]
6	5914	5	2019-09-06 18:00:20	2019-11-20 18:41:13
7	6615	5	2019-11-20 18:41:13	2021-04-12 15:45:29

Задача 6

Определите, когда был бы следующий check_in для каждого renter_id

```
SELECT  
    booking_id, renter_id, check_in_date,  
    LEAD(check_in_date) OVER (PARTITION BY renter_id ORDER BY  
    check_in_date) AS next_check_in  
FROM bookings;
```

	booking_id integer	renter_id integer	check_in_date timestamp without time zone	next_check_in timestamp without time zone
1	10796	1	2021-01-29 23:23:30	[null]
2	1526	2	2018-06-04 19:15:54	[null]
3	14937	3	2022-04-07 15:43:27	[null]
4	2238	4	2018-08-16 20:27:52	2021-08-21 17:09:52
5	12749	4	2021-08-21 17:09:52	[null]
6	5914	5	2019-09-06 18:00:20	2019-11-20 18:41:13
7	6615	5	2019-11-20 18:41:13	2021-04-12 15:45:29

Агрегатные функции и оконные

Характерно как для оконных, так и агрегатных функций:	Агрегатные функции отличаются от оконных функций в следующем:	Оконные функции отличаются от агрегатных функций в следующем:
• Работают с множеством строк • Вычисляют агрегатные величины • Группируют или секционируют данные по одному или нескольким столбцам	• Использование GROUP BY для определения множества строк для агрегации • Группировка строк на основе значений столбца • Сворачивание строк в единственную строку для каждой определенной группы	• Использование OVER() вместо GROUP BY для определения множества строк • Использование большего числа функций в дополнение к агрегатным, например: RANK(), LAG(), LEAD() • Могут группировать строки по их рангу, процентилю и т.д. в дополнение к значениям столбца • Не сворачивают строки в единственную строку на группу • Могут использовать скользящую рамку окна на основе текущей строки

Подзапросы

1. Условия выборки на основе вычисленных значений

Подзапросы позволяют фильтровать записи основной таблицы на основе агрегированных значений, вычисленных из той же таблицы или из связанных таблиц. Например, можно найти номера, цена которых выше средней по всем номерам нашей гостиницы.

	room_number integer	type_name text	price_per_night integer
1	1	Люкс	7460
2	9	Люкс	10000
3	12	Люкс	11327
4	18	Люкс	10632
5	19	Президентский	22964

Подзапросы

1. Условия выборки на основе вычисленных значений

Подзапросы позволяют фильтровать записи основной таблицы на основе агрегированных значений, вычисленных из той же таблицы или из связанных таблиц. Например, можно найти номера, цена которых выше средней по всем номерам нашей гостиницы.

```
SELECT room_number, type_name, price_per_night
FROM rooms
WHERE price_per_night > (
    SELECT AVG(price_per_night) as mean_price FROM rooms
)
```

	room_number integer	type_name text	price_per_night integer
1	1	Люкс	7460
2	9	Люкс	10000
3	12	Люкс	11327
4	18	Люкс	10632
5	19	Президентский	22964

Подзапросы

2. Извлечение данных, которые должны соответствовать результатам другого запроса

Выборка данных, основанная на соответствии или различии с данными, полученными из другого запроса. Например, можно выбрать все бронирования, которые стоили больше средней стоимости заказа.

	booking_id integer	renter_id integer	room_number integer	check_in_date timestamp without time zone	check_out_date timestamp without time
1	2	8148	45	2018-01-01 14:34:05	2018-01-04 08:19:05
2	5	8681	39	2018-01-01 18:21:29	2018-01-05 07:31:29
3	7	9371	42	2018-01-01 19:14:53	2018-01-04 07:24:53
4	8	3314	20	2018-01-01 19:21:02	2018-01-05 10:16:02
5	15	4913	49	2018-01-02 15:55:37	2018-01-05 07:04:37

Подзапросы

2. Извлечение данных, которые должны соответствовать результатам другого запроса

Выборка данных, основанная на соответствии или различии с данными, полученными из другого запроса. Например, можно выбрать все бронирования, которые стоили больше средней стоимости заказа.

```
SELECT
    booking_id, renter_id, room_number, check_in_date, check_out_date
FROM bookings
WHERE booking_id IN (
    SELECT booking_id
    FROM payments
    WHERE amount_paid > (SELECT AVG(amount_paid) FROM payments)
)
```

	booking_id integer	renter_id integer	room_number integer	check_in_date timestamp without time zone	check_out_date timestamp without time
1	2	8148	45	2018-01-01 14:34:05	2018-01-04 08:19:05
2	5	8681	39	2018-01-01 18:21:29	2018-01-05 07:31:29
3	7	9371	42	2018-01-01 19:14:53	2018-01-04 07:24:53
4	8	3314	20	2018-01-01 19:21:02	2018-01-05 10:16:02
5	15	4913	49	2018-01-02 15:55:37	2018-01-05 07:04:37

Подзапросы

3. Использование результата подзапроса в качестве источника данных

Подзапросы могут использоваться в FROM-секции для создания временной таблицы, которая затем используется для дальнейшего анализа или как часть более крупного запроса. Это позволяет выполнять сложные агрегации и анализы, которые были бы сложны или невозможны с использованием только основных таблиц. Выберем платежи, которые были в свой день самыми дорогими и самыми дешевыми.

	payment_date timestamp without time zone 🔒	payment_id integer 🔒	amount_paid integer 🔒
1	2017-12-17 00:00:00	8	25329
2	2017-12-19 00:00:00	7	14644
3	2017-12-20 00:00:00	16	2800
4	2017-12-21 00:00:00	17	15600
5	2017-12-21 00:00:00	3	5080
6	2017-12-22 00:00:00	9	1600
7	2017-12-25 00:00:00	82	5080

Подзапросы

3. Использование результата подзапроса в качестве источника данных

Подзапросы могут использоваться в FROM-секции для создания временной таблицы, которая затем используется для дальнейшего анализа или как часть более крупного запроса. Это позволяет выполнять сложные агрегации и анализы, которые были бы сложны или невозможны с использованием только основных таблиц. Выберем платежи, которые были в свой день самыми дорогими и самыми дешевыми.

```
SELECT payment_date, payment_id, amount_paid
FROM (
    SELECT
        payment_id,
        payment_date,
        amount_paid,
        MAX(amount_paid) OVER (PARTITION BY payment_date) as max_amount,
        MIN(amount_paid) OVER (PARTITION BY payment_date) as min_amount
    FROM payments
) as payments_agg
WHERE amount_paid = max_amount OR amount_paid = min_amount
ORDER BY payment_date;
```

	payment_date timestamp without time zone 🔒	payment_id integer 🔒	amount_paid integer 🔒
1	2017-12-17 00:00:00	8	25329
2	2017-12-19 00:00:00	7	14644
3	2017-12-20 00:00:00	16	2800
4	2017-12-21 00:00:00	17	15600
5	2017-12-21 00:00:00	3	5080
6	2017-12-22 00:00:00	9	1600
7	2017-12-25 00:00:00	82	5080

Подзапросы

4 . Использование результата подзапроса в условиях внутри SELECT

Мы можем использовать для этого подзапросы, которые возвращают число или столбец.

	room_number integer	price_per_night integer	room_price_category text
1	1	7460	Дорогой номер
2	2	3100	Дешевый номер
3	3	1500	Дешевый номер

Подзапросы

4 . Использование результата подзапроса в условиях внутри SELECT

Мы можем использовать для этого подзапросы, которые возвращают число или столбец.

```
SELECT
    room_number,
    price_per_night,
    CASE
        WHEN price_per_night > (
            SELECT AVG(price_per_night) FROM rooms
        ) THEN 'Дорогой номер'
        ELSE 'Дешевый номер'
    END as room_price_category
FROM rooms;
```

	room_number  integer	price_per_night  integer	room_price_category  text
1	1	7460	Дорогой номер
2	2	3100	Дешевый номер
3	3	1500	Дешевый номер

Подзапросы

5 . Использование результата подзапроса в условии HAVING

Выведем сумму покупок клиентов за каждый день, но только в те дни, когда эта сумма была выше средней.

	payment_date timestamp without time zone 	income bigint 
1	2019-08-20 00:00:00	124798
2	2018-02-07 00:00:00	180603
3	2021-09-11 00:00:00	160884
4	2021-11-11 00:00:00	127850
5	2021-03-08 00:00:00	413055

Подзапросы

5 . Использование результата подзапроса в условии HAVING

Выведем сумму покупок клиентов за каждый день, но только в те дни, когда эта сумма была выше средней.

```
SELECT payment_date, SUM(amount_paid) as income
FROM payments
GROUP BY payment_date
HAVING SUM(amount_paid) > (
    SELECT AVG(income) FROM (
        SELECT
            SUM(amount_paid) as income
        FROM payments
        GROUP BY payment_date
    ) income_agg
);
```

	payment_date timestamp without time zone 	income bigint 
1	2019-08-20 00:00:00	124798
2	2018-02-07 00:00:00	180603
3	2021-09-11 00:00:00	160884
4	2021-11-11 00:00:00	127850
5	2021-03-08 00:00:00	413055

Спасибо за внимание!